

```

|__pack
4 directories, 1 file
objects git: (main)

```

When I did *git add*, a new object was created. As described above, the first two characters are used as the directory and the rest as file names. To check for the content and type of object, there is a simple command – *git cat-file*.

```

objects git: (main) git cat-file -t
ec7ff284664263109178b5ea42eadfef0fa8e8e2
blob
objects git:(main) git cat-file -s
ec7ff284664263109178b5ea42eadfef0fa8e8e2
27
objects git: (main) git cat-file -p
ec7ff284664263109178b5ea42eadfef0fa8e8e2
Adding first line to file1
objects git: (main)

```

Here we can see that a blob object has been created for *file1.txt*. The size of this object is 27 bytes, and the content is 'Adding first line to file1' which is the same as what we added in the file.

Trees: For each directory in a repository, Git maintains a tree. This tree contains blob objects for all the files present in that directory. Since a directory can have multiple sub-directories inside it, a tree can also have multiple trees representing each sub-directory. The object ID of the tree object is also a SHA-1 hash value. A root tree represents a Git repository only; therefore, a root tree contains all the content of the Git repository.

As shown in Figure 1, I have committed the changes added in the previous section. Due to this a new object for tree and commit are also created.

```

objects git:(main) git cat-file -t
a3191707737bcf9357156ed70f879965e7795f69
tree
objects git:(main) git cat-file -p
a3191707737bcf9357156ed70f879965e7795f69
100644 blob ec7ff284664263109178b5ea42eadfef0fa8e8e2 file1.
txt

```

Here I have checked the content of the tree object. This tree is a root tree of the repository as of now. As shown, a tree contains a reference for the *file1.txt* blob object. If there are more directories in the repository, the tree will store the references to trees created for those sub-directories.

Figure 2 shows how tree and blob objects are related.

Commit: The third object created along with blob and tree objects is a commit. A commit is a snapshot of time. It

```

→ git_tutorial_dir git:(main) * git commit -m "Adding first commit"
[main (root-commit) ac6a979] Adding first commit
1 file changed, 1 insertion(+)
create mode 100644 file1.txt
→ git_tutorial_dir git:(main) cd .git/objects
→ objects git:(main) tree
.
├── a3
│   └── 191707737bcf9357156ed70f879965e7795f69
├── ac
│   └── 6a9797c263b7582ef2afb6dca81a4934f5c89c
├── ec
│   └── 7ff284664263109178b5ea42eadfef0fa8e8e2
└── info
    └── pack
6 directories, 3 files

```

Figure 1: Git commit

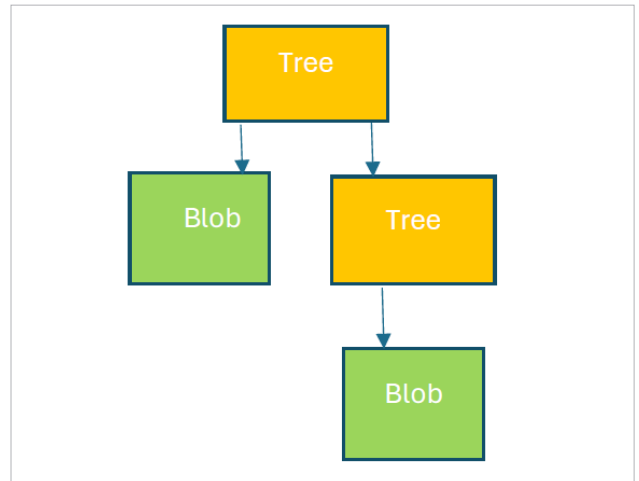


Figure 2: Tree and blob objects

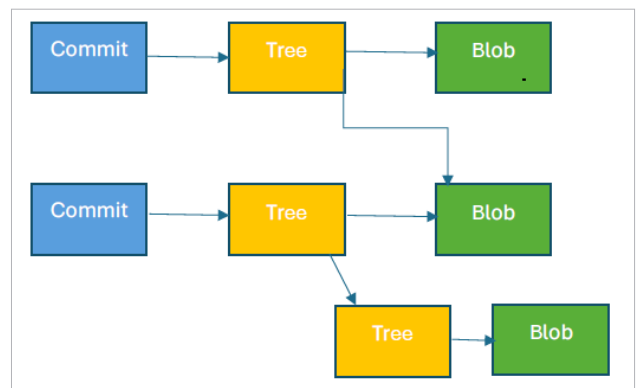


Figure 3: Commit

stores the object ID of the root tree only. Along with that it contains details such as the committer name, time stamp of commit, parent commit, and more. Its object ID is also a SHA-1 hash value.

```

→ objects git: (main) git cat-file -t
ac6a9797c263b7582ef2afb6dca81a4934f5c89c
commit

```